

Class XII | Unit 1 | weight 10%

System development life cycle

Board weight about 10% | 15 theory periods

A system is a set of parts working toward a purpose. Building a software system properly means moving through agreed phases, with a named person responsible for each kind of work.

What you should be able to do

- Define a system and the system development life cycle, and say why the cycle matters.
- Describe the roles of the project manager, analyst, architect, developers, testers, operations and support.
- Explain planning, requirements, design, implementation, testing, deployment and maintenance.
- Distinguish functional and non-functional requirements, and describe an SRS, bugs, errors and an SLA.

Path through the lecture

- 1 Use one system the class knows, such as a college library, and walk it through every phase.
- 2 Put the seven roles on the board beside the phase each role leads.
- 3 Write three functional and three non-functional requirements for that library before defining the SRS.

1.1 Systems and the people who build them

System. A set of interrelated components that work together to achieve a purpose.

System development life cycle. The ordered set of phases through which a software system is planned, built, tested, delivered and kept in service.

The cycle matters because a college that starts coding on the first day often builds the wrong system. Requirements that are written down can be checked. A design can be reviewed before money is spent on programming. Testing finds faults before students meet them. Maintenance keeps the system useful after delivery. The cycle also tells a team who is responsible for each kind of decision.

Stakeholders in the cycle

Role	Leads this work
Project manager	Plan, schedule, cost and coordination of the team
Business analyst	Finding and writing what the organisation needs
System architect	The overall design and structure of the solution
Software developer	Writing the programs
Tester	Checking that the system meets the requirements and reporting faults
Operational team	Putting the system into real use and running it
Production support	Looking after the system after it is live, including repairs and small changes

1.2 The phases



Maintenance feeds new requirements back into planning.

Figure 1.1 The phases used in this course. Maintenance sends new needs back into planning.

Planning and requirements

Problem identification names what is wrong or missing. A library may be losing track of which student has which book. **Planning** sets the scope, the time and the people. **Feasibility** asks whether the system should be built at all.

Feasibility questions

Kind	The question
Technical	Can it be built with the hardware, software and skills available?
Economic	Do the benefits justify the cost?
Operational	Will the librarians and students actually be able to use it?
Schedule	Can it be ready for the term in which it is needed?

Requirement gathering collects what the system must do, from interviews, existing forms and observation. A **functional requirement** is a behaviour: the librarian can issue a book to a member, search by title, and record a return. A **non-functional requirement** is a quality: the search should answer within a few seconds, only a librarian can delete a member, and the screens should be readable on the lab monitors.

Software requirements specification. The document that records what the system must do and the constraints on how well it must do it. Designers and testers work from this document.

An SRS usually contains an introduction, a description of the users, the functional requirements, the non-functional requirements, the data that must be stored, and the interfaces with other systems. It is the agreement between the college and the builders.

Design, build, test, deliver, maintain

Design decides the structure of the software, the data and the screens. An **algorithm** is a finite, ordered, unambiguous set of steps that solves a problem. A **flowchart** is a diagram of those steps. Use an oval for start and stop, a parallelogram for input or output, a rectangle for a process, a diamond for a decision, and arrows for the order.

Implementation is the coding of the design in a language such as C++. **Testing** runs the system to see whether it meets the SRS. A **bug** is a defect in the software. An **error** is a mistake or a wrong result: a person may write a wrong formula (a human error), the program may contain a defect (a bug), and the user may then see a wrong total (a failure or an incorrect result). In answers, define a bug as a fault in the program and an error as a wrong step or a wrong result, and give one example of each.

Deployment puts the finished system into the place where it will be used: the library computers, the real data and the trained staff. **Maintenance** repairs faults, adapts the system when rules change, and improves it. A **service level agreement** is a written commitment between the provider and the customer about the service, for example the hours the system will be available and how quickly a serious fault will be attended to.

Exam point

An 8-mark SDLC answer that only lists five headings will look thin. For each phase write one sentence of work and name the role that leads it. Use the library as the running example.

Key terms

Term	Meaning to write in an answer
System	Interrelated parts working for a purpose.
SDLC	The phases of planning, building, testing, delivering and maintaining software.
Feasibility	A study of whether the proposed system can and should be built.
Functional requirement	A behaviour the system must perform.
Non-functional requirement	A quality or constraint, such as speed or security.
SRS	The specification of requirements that guides design and testing.
Algorithm	A finite sequence of clear steps that solves a problem.
SLA	An agreement on the level of service to be provided.

Bring it together

- The life cycle reduces the chance of building the wrong system.
- Analysts gather requirements, architects design, developers code, testers test, operations deploy, and support maintains.
- Functional requirements say what the system does. Non-functional requirements say how well.
- A bug is a defect in the software. An SLA states the service the live system must provide.

Do this in the practical

- 1 Write six requirements for a college library system: three functional and three non-functional. Draw a flowchart to issue a book.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A functional requirement describes:

- A. What the system does
- B. Only the colour of the case
- C. The brand of keyboard
- D. A topology

2. The SRS is mainly used so that:

- A. Design and test have a written set of requirements
- B. RAM is refreshed
- C. A cable is crimped
- D. A pointer is dereferenced

3. Which role leads the writing of program code?

- A. Software developer
- B. Only the projector
- C. The CMOS battery
- D. The barcode

4. Economic feasibility asks whether:

- A. The benefits justify the cost
- B. The screen is an LCD
- C. The language is C++
- D. The bus is unidirectional

5. A diamond in a flowchart means:

- A. A decision
- B. A start symbol only
- C. A database table
- D. A wireless cell

6. An SLA belongs most naturally to:

- A. Support of a live system
- B. Erasing an EPROM
- C. Declaring an int
- D. A simplex link only

7. Maintenance happens:

- A. After the system is in use, and can lead to new requirements
- B. Only before the problem is identified
- C. Only inside the ALU
- D. Only on paper flowcharts

8. A bug is:

- A. A defect in the software
- B. A network topology
- C. A file attribute
- D. A normal form

Answer key

Q	Answer	Why this is the answer
1	A	Functional requirements are behaviours, such as issuing a book.
2	A	The specification is the agreed description of the system to be built.
3	A	Developers implement the design by coding.
4	A	Economic feasibility compares cost and benefit.
5	A	A diamond is the decision symbol. An oval is start or stop.
6	A	A service level agreement sets the service expected after deployment.
7	A	Maintenance looks after the live system and often starts another cycle.

Q	Answer	Why this is the answer
8	A	A bug is a fault in the program that can cause wrong behaviour.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define the system development life cycle.

Model answer

The system development life cycle is the set of phases through which a system is planned, designed, built, tested, deployed and maintained. It matters because each phase checks the work of the one before it, so the team is less likely to code a system that does not meet the need.

Q2. Differentiate functional and non-functional requirements.

Model answer

A functional requirement states a behaviour, for example that a librarian can record the return of a book. A non-functional requirement states a quality or constraint, for example that the record is saved securely and the screen responds promptly. Both belong in the SRS.

Q3. What is feasibility? Name two kinds.

Model answer

Feasibility is the study of whether a proposed system should be built. Technical feasibility asks whether the tools and skills exist. Economic feasibility asks whether the benefit justifies the cost. Operational and schedule feasibility are the other two kinds used in class.

Q4. Differentiate an algorithm and a flowchart.

Model answer

An algorithm is a written sequence of finite, clear steps that solves a problem. A flowchart is a diagram of those steps, using standard symbols such as a rectangle for a process and a diamond for a decision. The flowchart shows the same logic in a form that is easy to follow on paper.

Q5. What is a service level agreement?

Model answer

A service level agreement is a commitment between a service provider and a customer about the level of service. It can state the hours a system will be available and the time allowed to respond to a serious fault.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain the phases of the software development life cycle with the role responsible for each.

- 1 Define system and SDLC, and give one reason the cycle is used.
- 2 Describe problem identification, planning, feasibility and requirements, and name the analyst and the project manager.
- 3 Describe the SRS, then design and the architect, including algorithm and flowchart.
- 4 Describe coding by developers, testing by testers, deployment by operations, and maintenance by production support.
- 5 Define bug and SLA in the last two phases, using a library example throughout.

Question 2. Explain how requirements are gathered and written for a college library system.

- 1 Name the users: librarian, member, administrator.
- 2 Give three functional requirements in complete sentences.
- 3 Give three non-functional requirements: speed, security, usability.
- 4 Say what an SRS contains and who uses it afterwards.
- 5 Mention feasibility briefly so the college does not specify a system it cannot run.